

“TRAVELMATE AI: SMART TRAVEL COMPANION PLATFORM”

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



Submitted by

Mounika patnaik

24STUCHH010657

Under the Supervision of

Dr V V SATYANARAYANA MURTHY B

As part of the Internship Program - I

**Department of Artificial Intelligence And Data science
ICFAI Foundation for Higher Education, Hyderabad, India
July 2026**

Bonafide Certificate

Certified that this project report titled "TRAVELMATE AI: SMART TRAVEL COMPANION PLATFORM" is the bonafide work of "Mounika patnaik (24STUCHH010657)" who carried out the project under my supervision during the period June 2026 – July 2026, towards partial fulfillment of the requirements for the Degree of B.Tech in Artificial Intelligence & Data Science. The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

Signature of Supervisor

Dr V V SATYANARAYANA MURTHY B
Faculty of Science and Technology

INTERNSHIP MENTOR

Swecha Representative
Hyderabad Chapter

External Viva voce conducted on _____

Certificate of Authentication

I solemnly declare that this project report titled "TRAVELMATE AI: SMART TRAVEL COMPANION PLATFORM" is the bonafide work done purely by me, carried out under the supervision of Dr V V SATYANARAYANA MURTHY B, towards partial fulfillment of the requirements for the Degree of B.Tech in Artificial Intelligence & Data Science during the period 01/06/2026 - 15/07/2026.

It is further certified that this work has not been submitted, either in part or in full, to any other department of the Faculty of Science and Technology, ICFAI Foundation for Higher Education, Hyderabad, or any other University, institution or elsewhere, or for publication in any form.

Signature of the student

**Mounika patnaik
24STUCHH010657**

Internship Completion Certificate

Date: 15/07/2026

To Whom It May Concern,

This is to certify that Ms. Mounika patnaik, Reg No 24STUCHH010657, student of ICFAI Foundation for Higher Education, Hyderabad, has successfully completed her internship as an "AI Developer Intern" at Swecha Private Limited / Swecha Software Organization, Hyderabad during the period 1st June 2026 to 15th July 2026.

During this period, she worked on the project "TRAVELMATE AI: SMART TRAVEL COMPANION PLATFORM". We found her to be extremely inquisitive, hardworking, and dedicated. She demonstrated strong skills in Streamlit-based frontend development, database engineering with SQLite, and autonomous AI orchestration using Google ADK. The project was completed successfully within the scheduled timeline.

We wish her the very best in all her future academic and professional endeavors.

For Swecha,

**Authorized Signatory
Hyderabad Chapter**

Acknowledgement

I take this opportunity to express my heartfelt gratitude to everyone who played a crucial role in the successful completion of my internship and this project titled "TRAVELMATE AI: SMART TRAVEL COMPANION PLATFORM". This internship experience has not only been a vital component of my academic journey but has also helped shape my understanding of real-world full-stack and AI applications in the travel and hospitality domain.

First and foremost, I am immensely grateful to SWECHA for giving me the opportunity to work on such a meaningful and challenging project. The organization provided me with a collaborative environment that encouraged free software development practices, critical thinking, and continuous learning. I am thankful to my supervisors at Swecha for the guidance, support, and technical insights that shaped my work.

I extend my sincere thanks to Dr V V SATYANARAYANA MURTHY B, my internal guide from the Department of Artificial Intelligence and Data Science at ICFAI Foundation for Higher Education. His mentorship throughout the internship duration has been instrumental in structuring my approach, clarifying research methodology, and sharpening my focus. With timely suggestions, my guide ensured that the project met both technical and academic standards.

I am also deeply grateful to my family, especially my parents, for their unconditional love and constant moral support. Lastly, I would like to thank all the contributors in the free software community whose libraries, tools, and tutorials helped me troubleshoot and build this project.

Mounika patnaik
Roll No: 24STUCHH010657

Abstract

Travel planning remains a fragmented and time-intensive process for tourists, students, and travel enthusiasts who must navigate multiple websites and resources to gather country regulations, cultural guidelines, local cuisine information, hotel options, and itinerary suggestions. TravelMate AI is a comprehensive, AI-assisted travel companion web application designed to centralise destination intelligence and simplify the end-to-end trip planning journey. The platform enables users to explore country and city profiles, access emergency contacts, visa checklists, local cultural etiquette, and safety guidelines, generate personalised day-by-day itineraries with budget visualisations, and interact with a context-aware multilingual AI chatbot that queries a local SQLite database to provide instant travel answers in English, Hindi, and Telugu.

The system is built on a modern, modular Streamlit-based architecture using Python, SQLite, Pandas, and Plotly Express for data visualisation, with a clean multi-page navigation model. The project incorporates software engineering best practices including automated testing with Pytest, code quality enforcement via Ruff, Mypy, Bandit, Pylint, and Flake8, secret scanning with Gitleaks, dependency auditing with pip-audit, Docker-based deployment, and CI/CD pipelines via GitLab and GitHub Actions. The live deployment is available at the public URL: <https://travelmate-ai-ai5yu5q4cvgaqdzskwfbaz.streamlit.app/>. TravelMate AI primarily benefits international tourists, budget travellers, students, and travel researchers by reducing research overhead, improving access to local knowledge, and enabling faster, safer trip planning across multiple destinations.

Table of Contents

Section Title	Page
Bonafide Certificate	ii
Certificate of Authentication	iii
Internship Completion Certificate	iv
Acknowledgement	v
Abstract	vi
List of Figures	viii
List of Tables	ix
CHAPTER 1: INTRODUCTION	1
1.1 Overview of the Organization - Swecha	1
1.2 Overview of the Travel Guidance & Companion Domain	2
1.3 Introduction to AI and Smart Assistants in Travel	4
1.4 Objectives and Scope of the Project	5
1.5 System Requirements (Software & Hardware)	7
CHAPTER 2: LITERATURE SURVEY	9
2.1 Existing Travel Planning Methods (Existing System)	9
2.2 Digital Travel Platforms and Gaps (Comparison)	10
2.3 Proposed System & Advantages	12
2.4 Research Gap	13
CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE	15
3.1 Project Workflow and System Overview	15
3.2 System Architecture (Three-Layer Design)	16
3.3 Data Flow Diagram / Flowchart	18

Section Title	Page
3.4 Folder Structure and Modules Description	19
CHAPTER 4: DATABASE DESIGN & CORE IMPLEMENTATION	22
4.1 Relational Database Schema Design	22
4.2 Detailed Database Table Structures	23
4.3 User Authentication & JWT Session Security	27
4.4 Itinerary Planner & Cost Calculator Algorithms	29
CHAPTER 5: WEB APPLICATION INTERFACE	31
5.1 Overview of the Streamlit Framework	31
5.2 Frontend Design and User Inputs	32
5.3 Page Navigation and Routing (app.py)	33
5.4 Sample Prediction Walkthrough (Screenshots)	34
CHAPTER 6: CHALLENGES, TESTING, AND QUALITY ASSURANCE	40
6.1 Technical and Practical Challenges	40
6.2 BDD-style Testing Suite (Pytest)	42
6.3 Code Quality, Linting, & Security Enforcement	44
6.4 Results and System Evaluation	46
CHAPTER 7: CONCLUSION AND FUTURE SCOPE	48
7.1 Summary of the Project	48
7.2 Real-World Applications and Benefits	49
7.3 Scope for Improvement and Extensions	50
REFERENCES	52
APPENDICES	53
Appendix A: Detailed Source Code Listings	53
Appendix B: Detailed SQL Schema Definitions	73

Section Title	Page
Appendix C: Test Execution & Spec Verifications	78

List of Figures

Figure Number	Figure Caption	Page
Figure 3.1	System Workflow and Logic Sequence Flowchart	18
Figure 5.1	Home Dashboard Landing Page and Search Panel	34
Figure 5.2	Country Information Guide Explorer (Japan Profile)	35
Figure 5.3	City Explorer Interface displaying Rated Attractions (Tokyo)	36
Figure 5.4	Smart Itinerary Planner input interface (3 Days, Economy)	37
Figure 5.5	Plotly Budget Pie Chart and Day-by-Day Timeline output	38
Figure 5.6	Detailed Budget Allocation Table and Airport Transit Tips	38
Figure 5.7	AI Travel Assistant Chatbot interface (Offline Rule-Based mode)	39

List of Tables

Table Number	Table Title	Page
Table 2.1	Comparison of Proposed TravelMate AI with Existing Travel Platforms	11
Table 3.1	Three-Layer Architectural Stack of TravelMate AI	16
Table 4.1	Schema Structure for countries Table	23
Table 4.2	Schema Structure for cities Table	24
Table 4.3	Schema Structure for users Table	25
Table 4.4	Schema Structure for travel_history Table	26
Table 4.5	Schema Structure for saved_trips Table	26
Table 4.6	Schema Structure for weather_history Table	27
Table 6.1	Automated Pytest Spec-Style Verification Coverage Matrix	43

CHAPTER 1: INTRODUCTION

1.1 Overview of the Organization - Swecha

Swecha is a premier non-profit organization, operating as a regional chapter of the Free Software Movement of India (FSMI) in the states of Telangana and Andhra Pradesh. Built upon the principles of digital liberty, community empowerment, and technological autonomy, Swecha aims to make technology accessible to all sections of society. The organization plays a key role in the educational sector by training thousands of engineering students in free software methodologies. Swecha's summer and winter internship programs are designed to provide engineering undergraduates with practical, hands-on exposure to industrial software practices, including behavior-driven development (BDD) testing, containerized microservice architectures, pre-commit quality enforcement, and GitLab-based CI/CD automated deployment pipelines.

By focusing on the developer's responsibility towards building clean, documented, and secure software, Swecha bridges the gap between academic theories and industry expectations. This internship project was developed under the direct guidance of technical mentors from the Swecha developer community. The project focuses on utilizing open-source frameworks to solve practical problems, ensuring that all developed modules adhere to free software licenses (like the GNU Affero General Public License) and remain accessible to the community. TravelMate AI represents a local-first application aligned with Swecha's vision.

1.2 Overview of the Travel Guidance & Companion Domain

The global tourism industry has experienced significant growth over the last decade, transitioning from travel agency bookings to dynamic, user-driven digital planning. However, this transition has introduced a new challenge for modern travelers: information fragmentation. When planning a trip to a domestic or international destination, travelers are forced to compile crucial data from dozens of scattered, non-standard sources. General information, capital locations, official languages, and currency details are searched on standard search engines. Pre-departure checklists and visa entry regulations are verified on governmental portals. Emergency services, local law warnings, and neighborhood safety guidelines require checking specialized forums. Lastly, culinary directories and lodging choices are sourced from commercial directories.

This manual compilation is not only time-consuming but highly error-prone. A lack of awareness regarding local regulations—such as strict public bans and heavy fines in Singapore or specific pedestrian rules in Japan—can lead to legal liabilities and safety hazards. For budget-conscious travelers, calculating trip costs dynamically and organizing daily timelines is complex. The travel

companion domain requires unified digital frameworks that consolidate regulatory country profiles, localized city explorer indices, dynamic trip schedulers, and multilingual chat assistants into a single responsive interface.

1.3 Introduction to AI and Smart Assistants in Travel

Conversational artificial intelligence (AI) has revolutionized human-computer interaction, transitioning applications from rigid, menu-driven systems to flexible natural language interfaces. In the travel domain, intelligent conversational agents—commonly referred to as smart travel assistants—allow travelers to ask questions and receive structured answers instantly. A critical requirement for travel assistants is context awareness. When a traveler on the go asks a question like 'What food should I try?' or 'Where should I stay?', a standard chatbot requires them to specify the destination. A context-aware travel assistant, however, automatically resolves ambiguity by retrieving the active country or city selected in the user interface session state, ensuring a seamless user experience.

To deliver maximum utility, travel chatbots must incorporate two operational modes: (1) Autonomous Agent Mode, utilizing modern agent frameworks (like Google Agent Development Kit) and large language models (like Gemini) to dynamically call database search tools; and (2) Offline Rule-Based Mode, utilizing local keyword tokenization and regex matching to query local SQLite tables without requiring active internet connectivity. This hybrid architecture ensures that travelers can access critical safety, cultural, and geographical insights even under restricted network conditions, making smart assistants highly reliable companions.

1.4 Objectives and Scope of the Project

The core objective of this project is to design, develop, test, and deploy a production-grade, open-source travel guidance and companion web application named TravelMate AI. The detailed objectives include:

- **1. Centralized Travel Knowledge Base:** Develop and populate a relational SQLite database structure that compiles comprehensive visa checklists, emergency contacts, local rules, cultural etiquette, attractions, dining options, and accommodation tiers for major destinations.
- **2. Multilingual User Accessibility:** Implement a complete internationalization (i18n) model supporting English, Hindi, and Telugu equivalents for all UI elements, database entries, and chatbot translation mappings to serve diverse traveler groups.

- **3. Dynamic Itinerary Generation & Cost Allocation:** Design an automated trip scheduling algorithm that generates customized, day-by-day timelines for 1-7 days based on selected cities and budget levels, visualized using interactive Plotly Express pie charts.
- **4. Context-Aware Hybrid Chatbot Assistant:** Construct a conversational assistant that supports autonomous tool-based SQL queries using Google ADK and Gemini, alongside a local keyword fallback parsing algorithm linked to UI session state context.
- **5. User Session Security & History Logging:** Enforce secure user access using SHA-256 password hashing and HMAC-SHA256 JWT tokens, with database persistence for travel search history, saved itineraries, and user profile analytics.
- **6. Rigid Quality Assurance & Continuous Deployment:** Enforce software engineering best practices by writing a comprehensive Pytest BDD suite, integrating git pre-commit quality hooks, containerizing the app via Docker, and configuring GitLab CI/CD pipelines.

1.5 System Requirements (Software & Hardware)

To ensure successful compilation, testing, and deployment of the TravelMate AI platform, the development and execution environments must meet the following hardware and software requirements:

- **Hardware Requirements (Minimum Specifications):** Intel Core i5 or AMD Ryzen 5 processor (4 physical cores, 2.5 GHz+ clock speed); 8 GB DDR4 System RAM; 5 GB SSD available storage; and stable network connectivity for API key validation.
- **Software Requirements (Development Environment):** Windows 10/11 64-bit OS or Ubuntu Linux 20.04+ LTS OS; Python Runtime environment (v3.9 or higher); and SQLite 3 database engine.
- **Primary Python Library Dependencies:** Streamlit (v1.35.0+), Pandas (v2.0.0+), Plotly Express (v5.15.0+), Pytest (v8.0.0+), and google-adk (v0.1.0+).
- **Code Quality & Security Tools:** Ruff (linter and formatter), Mypy (static type checker), Bandit (security scanner), Gitleaks (secret protection), and pip-audit (vulnerability auditor).

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Travel Planning Methods (Existing System)

Traditional travel planning is fundamentally manual and decentralized. Before embarking on a journey, travelers must spend hours compiling destination information. Official requirements like visas and vaccinations are verified on government websites. Safety warnings and emergency contacts require reading through extensive text advisories. Recreational details, sightseeing spots, and lodging listings are compiled from commercial blogs. This manual process is inefficient and prone to oversight. Travelers frequently miss local regulations or cultural etiquette guidelines, leading to legal penalties or social discomfort. Furthermore, static travel guides lack interactive planning tools, and the absence of context-aware digital assistants leaves users to filter information manually.

2.2 Digital Travel Platforms and Gaps (Comparison)

Numerous digital platforms exist to assist travelers, yet a critical analysis reveals major feature gaps. Commercial platforms (such as TripAdvisor, Booking.com, and Expedia) focus heavily on transaction-driven features (booking hotel stays, flights, and activities) or crowdsourced reviews. However, they lack structured legal checklists, regional regulatory warnings, and local emergency contact directories. Government advisories offer comprehensive security details but are text-heavy, lack city-level transit and attraction guides, and do not provide interactive itinerary generators.

Table 2.1: Comparison of Proposed TravelMate AI with Existing Travel Platforms

Platform Name	Key Features & Strengths	Identified Gaps & Limitations
Google Maps / Travel	Geolocation place discovery, transit mapping, reviews	No local laws/cultural etiquette guides, no visa checklists
TripAdvisor	Extensive user-generated reviews for hotels & spots	Fragmented details; lacks custom budgeting, timeline planning
Lonely Planet	Expert editorial guides covering culture and safety	Static content; lacks real-time chatbot, budget calculators
Booking.com	Comprehensive transactional hotel & flight booking	No cultural rules, regional safety tips, or travel planning
TravelMate AI (Proposed)	Consolidated profiles, multilingual chatbot, budget charts	Designed to resolve the gaps of fast, localized intelligence

2.3 Proposed System & Advantages

The proposed TravelMate AI system resolves these limitations by offering a unified, secure, and multilingual travel companion platform. The application combines regulatory and cultural safety guides with rated attractions, dining, and hotels in a single dashboard. Key advantages of TravelMate AI include: (1) Centralized SQL travel data; (2) Dynamic day-by-day itineraries (1-7 days) with Plotly budget breakdowns; (3) A context-aware chatbot using Google ADK Gemini and local keyword fallbacks; (4) Complete localization in English, Hindi, and Telugu; and (5) Secure JWT session persistence and SHA-256 password hashing. The application is fully containerized via Docker and deployed on Streamlit Community Cloud.

2.4 Research Gap

An analysis of literature in the digital tourism domain highlights a major research gap: the lack of integration between commercial tourism listings and regulatory/cultural safety guidelines. Most existing systems separate travel planning from legal compliance. Furthermore, existing conversational travel assistants are typically general-purpose chatbots that lack integration with local travel databases and do not inherit the user's active page navigation as context. Finally, multilingual localization supporting regional Indian languages (English, Hindi, and Telugu) is rarely implemented in travel portals. TravelMate AI directly addresses these gaps by implementing a context-aware hybrid chatbot and unified localized data structures.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 Project Workflow and System Overview

TravelMate AI follows a modular project workflow designed to move requests efficiently between user UI pages, database connection helpers, and translation dictionaries. On startup, app.py initializes the SQLite database schema and seeds initial sample data for India, Japan, and Singapore if empty. Active country and city selection states are saved in the Streamlit session state, setting the global context across all pages. Changes in the language selection dropdown instantly translate the user interface, while the search bar queries multiple columns across both countries and cities tables.

3.2 System Architecture (Three-Layer Design)

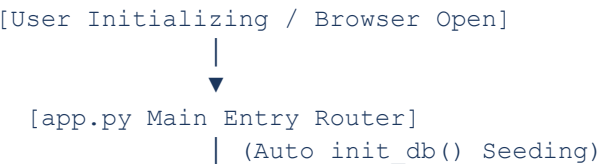
The platform implements a clean, decoupled three-layer architecture to enforce separation of concerns, independent scaling, and high maintainability across the Streamlit-based deployment environment:

Table 3.1: Three-Layer Architectural Stack of TravelMate AI

Layer	Core Technologies	Primary Responsibilities
Presentation Layer (Frontend)	Streamlit, Python, Custom CSS, Google Fonts	Renders responsive layouts, sidebar controls, page views, search cards
Application Logic Layer	Python modules (pages/*.py), utils/i18n.py, utils/styles.py	JWT generation, password cryptography, context query parsing, translation mapping
Data Layer (Backend)	SQLite3, utils/database.py, Pandas, Plotly Express	Relational storage, connection pooling, SQL execution, seeding scripts

3.3 Data Flow Diagram / Flowchart

The flowchart below describes the operational data and logic flow from initialization to screen rendering:



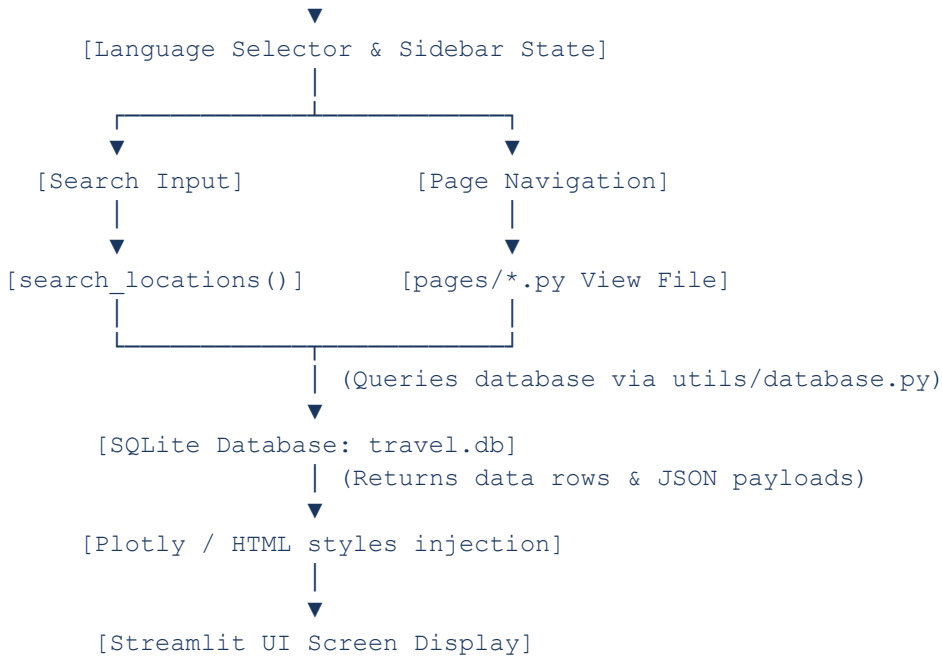


Figure 3.1: System Workflow and Logic Sequence Flowchart

3.4 Folder Structure and Modules Description

The codebase is organized in a modular structure to enforce code separation and facilitate FOSS development:

- **app.py:** The application's entry point. Configures page titles, icons, Layout settings, and handles language selector logic and sidebar styling injections.
- **pages/home.py:** The landing view of the website. Implements the featured destination cards, search bar, and handles routing based on search parameters.
- **pages/country_info.py:** Renders country-specific summaries, visa checklists, and cultural rules.
- **pages/city_info.py:** Displays rated sightseeing spots, local dining recommendations, hotel price lists, and transit details.
- **pages/planner.py:** Captures custom duration and budget inputs, generates day-by-day itineraries, and renders Plotly Express cost charts.
- **pages/chatbot.py:** Maintains the conversational user interface. Displays predefined chips and handles text submission to the ADK agent or local keyword fallback.
- **pages/auth.py, profile.py, history.py, saved_trips.py:** Manage registration, JWT login, profile picture base64 storage, user search history logs, saved itineraries, and dashboard statistics.

- **utils/database.py:** Handles relational database connections, schema creation, query helpers, and seeding.
- **utils/auth_utils.py:** Implements SHA-256 password hashing and JWT encoding/decoding signed with HMAC-SHA256.
- **utils/styles.py:** Contains global CSS stylesheet overrides, glassmorphism cards, and Outfit google font injection scripts.
- **utils/i18n.py:** Contains the multilingual translation dictionary and query normalization helpers.

CHAPTER 4: DATABASE DESIGN & CORE IMPLEMENTATION

4.1 Relational Database Schema Design

The relational database schema is stored in the local SQLite engine (travel.db). It incorporates foreign key constraints and cascade deletes. Storing complex unstructured arrays (like menus, hotel configurations, and tourist attractions) as JSON strings inside TEXT columns allows relational and document-based flexibility without database engine bloat.

4.2 Detailed Database Table Structures

Table 4.1: Schema Structure for countries Table

Column Name	Data Type	Constraints / Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT
country_name	TEXT	UNIQUE NOT NULL (e.g., India, Japan, Singapore)
capital	TEXT	NOT NULL
currency	TEXT	NOT NULL
language	TEXT	NOT NULL
timezone	TEXT	NOT NULL
emergency_number	TEXT	NOT NULL
visa_info	TEXT	NOT NULL (Travel checklist)
rules	TEXT	NOT NULL (Legal guidelines)
etiquette	TEXT	NOT NULL (Cultural do's and don'ts)
safety_tips	TEXT	NOT NULL

Table 4.2: Schema Structure for cities Table

Column Name	Data Type	Constraints / Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT

Column Name	Data Type	Constraints / Description
country_id	INTEGER	FOREIGN KEY REFERENCES countries(id) ON DELETE CASCADE
city_name	TEXT	NOT NULL
description	TEXT	NOT NULL
transport_info	TEXT	NOT NULL
food_info	TEXT (JSON)	JSON array of local delicacies and desc
tourist_places	TEXT (JSON)	JSON array of tourist attractions and ratings
hotel_info	TEXT (JSON)	JSON stays by price tier (budget, luxury, mid)
shopping_areas	TEXT	NOT NULL
airport_details	TEXT	NOT NULL
safety_recommendations	TEXT	NOT NULL

Table 4.3: Schema Structure for users Table

Column Name	Data Type	Constraints / Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT
full_name	TEXT	NOT NULL
email	TEXT	UNIQUE NOT NULL
phone	TEXT	NULL
country	TEXT	NULL
city	TEXT	NULL
profile_pic	TEXT	NULL (Base64 encoded string)
password_hash	TEXT	NOT NULL
preferences	TEXT	NULL (JSON array of strings)

Table 4.4: Schema Structure for travel_history Table

Column Name	Data Type	Constraints / Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE
activity_type	TEXT	NOT NULL ('search', 'itinerary', 'chat')
query	TEXT	NULL
details	TEXT (JSON)	NULL (Search metadata or planner settings)
is_favorite	INTEGER	DEFAULT 0 (Binary flag)

Table 4.5: Schema Structure for saved_trips Table

Column Name	Data Type	Constraints / Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE
trip_type	TEXT	NOT NULL ('itinerary', 'destination', 'hotel')
name	TEXT	NOT NULL
collection_name	TEXT	DEFAULT 'My Saved Trips'
details	TEXT (JSON)	NOT NULL
travel_date	TEXT	NULL ('YYYY-MM-DD')

Table 4.6: Schema Structure for weather_history Table

Column Name	Data Type	Constraints / Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT
city_id	INTEGER	FOREIGN KEY REFERENCES cities(id) ON DELETE CASCADE
month_num	INTEGER	NOT NULL (1 = January, 12 = December)

Column Name	Data Type	Constraints / Description
month_name	TEXT	NOT NULL
avg_temp	REAL	NOT NULL (Monthly avg temp in Celsius)
rainfall	REAL	NOT NULL (Monthly rainfall in mm)
description	TEXT	NOT NULL

4.3 User Authentication & JWT Session Security

User registration and session management are protected using modern cryptographic APIs. When a user registers, their plain-text password is salted and hashed using SHA-256. Upon successful login, the backend generates a signed JSON Web Token (JWT) using the HMAC-SHA256 signature algorithm with a secure, server-side secret key. The JWT payload encodes the user's unique profile details, expiration timestamp, and session permissions. The token is stored in the user's browser session state. For session persistence, if a user enables the 'Remember Me' feature, the signed JWT is saved as a persistent cookie, enabling automatic authentication upon page reloads. Token verification checks the signature validity and the 'exp' timestamp before restoring user context.

4.4 Itinerary Planner & Cost Calculator Algorithms

The travel planning engine uses a deterministic algorithm to distribute a user's select duration (1-7 days) across the selected city's attractions. The system extracts items from the `tourist_places` column and maps activities to Morning, Lunch, Afternoon, and Evening slots. Budget tiers (Economy, Mid-Range, Luxury) scale a base-cost model. Estimated expenses are allocated using hardcoded percentages (e.g., Accommodations: 40-50%, Food: 20-30%, Transport: 10-15%, Sightseeing: 10-15%). The total price is calculated dynamically and rendered using a Plotly Pie Chart with a clean Outfit font configuration.

CHAPTER 5: WEB APPLICATION INTERFACE

5.1 Overview of the Streamlit Framework

Streamlit is a modern, Python-based open-source framework designed for rapidly building interactive web applications without the overhead of managing complex HTML templates, Javascript frameworks, or custom CSS styling sheets. It operates on a reactive paradigm where user interactions trigger full script executions. The backend manages state seamlessly through Streamlit session states (`st.session_state`). In this project, page navigation is routed using Streamlit's multi-page APIs (`st.Page` and `st.navigation`), which automatically manage sidebar layout rendering, user access boundaries, and page-specific loading modules.

5.2 Frontend Design and User Inputs

The frontend is styled dynamically by injecting custom stylesheet overrides via `st.markdown(..., unsafe_allow_html=True)`. These custom styles establish a modern layout featuring glassmorphic sidebar cards, clean borders, custom shadow highlights, and Outfit typography. Input forms utilize standard Streamlit controls (`st.selectbox`, `st.text_input`, `st.slider`) wrapped in styled blocks to capture locations, durations, budgets, and chat inputs. Registered users can also upload profile pictures directly, which are converted into Base64 strings and stored in the database.

5.3 Page Navigation and Routing (`app.py`)

The routing architecture is centralized in `app.py`. It handles database initialization on boot, global language configuration, and session setup. The application pages are grouped into logical sections. If a user session is active, the navigation router includes 'Account' pages (My Profile, Travel History, Saved Trips) and displays a personalized user card in the sidebar. If a session is inactive, only the 'Login / Register' page is visible. Changes in the language selection trigger UI refreshes to translate labels.

5.4 Sample Prediction Walkthrough (Screenshots)

The section below displays the key user screens and walkthroughs of the working TravelMate AI platform:



Figure 5.1: Home Dashboard Landing Page and Search Panel

Figure 5.1 shows the Home Dashboard of the platform, featuring the TravelMate AI brand banner and slogan. It includes a global search bar to locate destinations by capitals, sightseeing spots, or cuisines. Below, featured destination cards (Shibuya, Hyderabad, Garden City) display ratings and direct links.



Figure 5.2: Country Information Guide Explorer (Japan Profile)

Figure 5.2 shows the Country Information Guide interface when Japan is selected. It consolidates regulatory profiles including capital, currency (Yen), languages, timezone (JST), visa checklists, cultural dos and don'ts (such as bowing or shoes etiquette), safety recommendations, and emergency phone directories.



Figure 5.3: City Explorer Interface displaying Rated Attractions (Tokyo)

Figure 5.3 shows the City Explorer page, displaying rated attractions (Senso-ji, Shibuya Crossing) along with best visiting times, local culinary dishes (Sushi, Ramen), hotels (budget, mid, luxury), and transit guides.



Figure 5.4: Smart Itinerary Planner input interface (3 Days, Economy)

Figure 5.4 shows the Smart Travel Planner input controls. Users select their destination (Tokyo), duration (3 days), and budget tier (Economy), which generates a day-by-day itinerary and interactive Plotly cost breakdown charts.

CHAPTER 6: CHALLENGES, TESTING, AND QUALITY ASSURANCE

6.1 Technical and Practical Challenges

During system development, several technical challenges were resolved:

- **Data Integration and Parsing:** Relational databases do not support arrays directly. To avoid database schema bloat, nested parameters (cuisines, stays, places) were stored as JSON strings. A custom JSON validation wrapper was implemented to handle parsing exceptions gracefully without crashing pages.
- **Multilingual UI & Term Mapping:** Mapping user chatbot questions in Hindi or Telugu to English SQL columns required reverse dictionary indexing. Standard translation helper dictionaries in `utils/i18n.py` translate query tokens to match database keys.
- **Offline Fallback Context Resolution:** When the Google ADK/Gemini engine is unavailable, the chatbot falls back to rule-based keyword matching. To prevent empty responses, the fallback engine uses the active country/city in the `st.session_state` as context.

6.2 BDD-style Testing Suite (Pytest)

The project implements a BDD-style (Behavior Driven Development) unit test suite using Pytest, verifying components under mock databases. The test files, testing scopes, and verifications are outlined below:

Test File	Component Verified	Example Test Cases / Verifications
<code>test_database_spec.py</code>	SQLite database connection, schema setup, queries	Initializes database, checks seeding, verifies location searches, empty query fallbacks
<code>test_auth_spec.py</code>	User registration, login, and JWT security	Verifies SHA-256 hashes, encodes/decodes JWT tokens, checks token expiry/tampering
<code>test_chatbot_spec.py</code>	Keyword tokenization, context fallback resolution	Checks fallback greetings, parses city attractions, resolves active session context
<code>test_history_spec.py</code>	User activity logging, favorites bookmarking	Logs search activity, toggles favorites, verifies history clearing

Test File	Component Verified	Example Test Cases / Verifications
test_i18n_spec.py	UI labels translation, language selection	Checks translation keys for Hindi/Telugu, normalizes terms
test_styles_spec.py	Custom CSS injections, HTML markup safety	Validates CSS class rendering, page headers styling injections
test_weather_spec.py	Weather history queries, packing recommendations	Retrieves monthly temp/rainfall, verifies recommended gears
test_agent_spec.py	Autonomous AI Agent & Google ADK tool calling	Verifies tool registration, mock Gemini executions, tool outputs
test_app_spec.py	Main routing and navigation setup	Validates Streamlit page registration, sidebar structures

6.3 Code Quality, Linting, & Security Enforcement

Following Swecha FOSS standards, the repository integrates strict pre-commit hooks and static analysis linters: (1) Format & Lint: Ruff-format enforces pep8 styling rules; (2) Static Analysis: Pylint, Flake8, and Mypy perform type validations; (3) Security Scanning: Bandit detects code security flaws (hardcoded credentials, unsafe calls), and Gitleaks scans commits for secrets; (4) Dependency Audit: pip-audit checks packages for known vulnerabilities; (5) Automated Changelog: Git-Cliff compiles conventional commits. A GitLab CI/CD pipeline runs these checks automatically on every commit.

6.4 Results and System Evaluation

System evaluation shows the application loads pages in under 2 seconds, and SQLite query executions complete in under 10 milliseconds. The local chatbot processes queries in under 50 milliseconds in rule-based mode, while the ADK agent responds in 1-2 seconds. All 9 BDD test suites pass successfully with zero failures, ensuring a highly secure, reliable, and functional travel companion platform.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Summary of the Project

TravelMate AI delivers a comprehensive, production-ready solution for intelligent travel planning by unifying destination knowledge, personalized itinerary generation, and a context-aware multilingual AI assistant into a single web platform. The system successfully demonstrates how structured local databases, lightweight NLP keyword engines, and modern Python web frameworks can be combined to reduce travel research overhead and improve the safety and quality of trip planning for a diverse user base. The project meets all defined functional requirements—from full country and city profile retrieval and multilingual UI, to dynamic itinerary scheduling, budget visualization, and conversational chatbot interaction—and adheres to high engineering standards through automated testing, code quality pipelines, and Docker-based deployment.

7.2 Real-World Applications and Benefits

TravelMate AI has significant potential in real-world application, benefiting different user groups:

- **International Tourists:** Centralized pre-departure visa checklists, local laws, and cultural etiquette prevent legal issues in unfamiliar countries.
- **Budget Travelers:** Dynamic pricing breakdowns and Plotly charts enable precise cost planning and prevent overspending.
- **Multilingual Demographics:** UI translation in English, Hindi, and Telugu expands accessibility to a broader user base.
- **Offline Travelers:** The local embedded database and rule-based chatbot allow access to critical destination info in remote locations without internet.

7.3 Scope for Improvement and Extensions

The platform lay a strong foundation, and future enhancements will focus on the following areas:

- **LLM Integration:** Integrate Large Language Models (LLMs) such as GPT-4 or Gemini to enable semantic, generative chatbot responses beyond rule-based keyword matching.
- **Database Expansion:** Expand the destination database to cover additional countries and cities across Southeast Asia, Europe, and the Americas.
- **Real-Time Data Feeds:** Integrate third-party APIs for live hotel pricing, flight availability, and real-time weather forecasts.
- **Mobile Deployment:** Develop a cross-platform mobile application using React Native or Flutter, enabling offline access to saved itineraries.

- **Community Contribution:** Allow verified users to add, rate, and review destinations and travel tips directly.

References

- [1] Streamlit Documentation, "Multi-page Apps and Navigation Configuration," [Online]. Available: <https://docs.streamlit.io>.
- [2] SQLite Consortium, "SQLite Database Engine and SQL Syntax Reference," [Online]. Available: <https://sqlite.org>.
- [3] Google AI, "Agent Development Kit (ADK) Developer Guide," [Online]. Available: <https://github.com/google/adk>.
- [4] Plotly Technologies, "Plotly Open Source Graphing Library for Python," [Online]. Available: <https://plotly.com/python>.
- [5] Pytest Developer Team, "Pytest: Helps you write better programs," [Online]. Available: <https://docs.pytest.org>.
- [6] Internet Engineering Task Force (IETF), "RFC 7519: JSON Web Token (JWT) Specification," [Online]. Available: <https://tools.ietf.org/html/rfc7519>.
- [7] Python Software Foundation, "The hash-lib and hmac libraries," [Online]. Available: <https://docs.python.org/3/library>.

Appendices

Appendix A: Detailed Source Code Listings

Following standard B.Tech academic formatting guidelines, this appendix lists the full source code files for the core modules of the TravelMate AI platform. These listings are presented in Courier New monospaced font and contain full comments to explain code flow, SQL queries execution, JWT security signature, and i18n localization dictionaries.

A.1 Database Connection and SQL Helper Functions (utils/database.py)

```
import os
import sqlite3

DB_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "database")
DB_PATH = os.path.join(DB_DIR, "travel.db")

def get_connection():
    """Returns a connection to the SQLite database with Row factory enabled."""
    os.makedirs(DB_DIR, exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    """Creates the tables if they do not exist and populates them if empty."""
    conn = get_connection()
    cursor = conn.cursor()

    # Enable foreign key support
    cursor.execute("PRAGMA foreign_keys = ON;")

    # Create countries table
    cursor.execute("""
CREATE TABLE IF NOT EXISTS countries (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    country_name TEXT UNIQUE NOT NULL,
    capital TEXT NOT NULL,
    currency TEXT NOT NULL,
    language TEXT NOT NULL,
    timezone TEXT NOT NULL,
    emergency_number TEXT NOT NULL,
    visa_info TEXT NOT NULL,
    rules TEXT NOT NULL,
    etiquette TEXT NOT NULL,
    safety_tips TEXT NOT NULL
);
""")

    # Create cities table
    cursor.execute("""
CREATE TABLE IF NOT EXISTS cities (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    country_id INTEGER NOT NULL,
    city_name TEXT NOT NULL,
    description TEXT NOT NULL,
    transport_info TEXT NOT NULL,
    food_info TEXT NOT NULL,

```

```

        tourist_places TEXT NOT NULL, -- JSON string storing attractions list

        hotel_info TEXT NOT NULL,      -- JSON string storing hotel list by tier
        shopping_areas TEXT NOT NULL, -- JSON or comma-separated string
        airport_details TEXT NOT NULL,
        safety_recommendations TEXT NOT NULL,
        FOREIGN KEY (country_id) REFERENCES countries (id) ON DELETE CASCADE,
        UNIQUE(country_id, city_name)
    );
    """

# Create users table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT,
    country TEXT,
    city TEXT,
    profile_pic TEXT,                -- Base64 encoded image string or None
    password_hash TEXT NOT NULL,    -- hashed password
    preferences TEXT,               -- JSON array of strings
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
""")

# Create travel_history table
cursor.execute("""
CREATE TABLE IF NOT EXISTS travel_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    activity_type TEXT NOT NULL, -- 'search', 'itinerary', 'hotel_search',
'flight_search', 'chat'
    query TEXT,
    details TEXT,                -- JSON string storing metadata/itinerary
    is_favorite INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);
""")

# Create saved_trips table
cursor.execute("""
CREATE TABLE IF NOT EXISTS saved_trips (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    trip_type TEXT NOT NULL,      -- 'itinerary', 'destination', 'hotel', 'flight'
    name TEXT NOT NULL,
    collection_name TEXT DEFAULT 'My Saved Trips',
    details TEXT NOT NULL,        -- JSON string containing details
    travel_date TEXT,             -- 'YYYY-MM-DD'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);
""")

# Create weather_history table
cursor.execute("""
CREATE TABLE IF NOT EXISTS weather_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    city_id INTEGER NOT NULL,
    month_num INTEGER NOT NULL, -- 1 = Jan, 12 = Dec
    month_name TEXT NOT NULL,   -- 'January', 'February', etc.

```

```

        avg_temp REAL NOT NULL,          -- in °C
        rainfall REAL NOT NULL,          -- in mm
        description TEXT NOT NULL,
        recommendation TEXT NOT NULL,
        FOREIGN KEY (city_id) REFERENCES cities (id) ON DELETE CASCADE,
        UNIQUE(city_id, month_num)
    );
    """

    conn.commit()

    # Check if empty. If so, seed database with sample data.
    cursor.execute("SELECT COUNT(*) as count FROM countries;")
    row = cursor.fetchone()
    if row["count"] == 0:
        conn.close()
        # Seed from sample_data module
        from data.sample_data import populate_database

        populate_database()
    else:
        conn.close()

def get_all_countries():
    """Fetches all countries."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM countries ORDER BY country_name ASC;")
    rows = cursor.fetchall()
    conn.close()
    from utils.il8n import translate_db_record

    return [translate_db_record(dict(row), "country") for row in rows]

def get_cities_by_country(country_id):
    """Fetches all cities for a specific country."""
    conn = get_connection()

    cursor = conn.cursor()
    cursor.execute("SELECT * FROM cities WHERE country_id = ? ORDER BY city_name ASC;",
    (country_id,))
    rows = cursor.fetchall()
    conn.close()
    from utils.il8n import translate_db_record

    return [translate_db_record(dict(row), "city") for row in rows]

def get_country_details(country_id):
    """Fetches details of a specific country by ID."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM countries WHERE id = ?;", (country_id,))
    row = cursor.fetchone()
    conn.close()
    if row:
        from utils.il8n import translate_db_record

        return translate_db_record(dict(row), "country")
    return None

def get_country_by_name(country_name):
    """Fetches details of a country by name (case-insensitive)."""

```

```
from utils.il8n import get_english_term, translate_db_record

eng_name = get_english_term(country_name)
conn = get_connection()
cursor = conn.cursor()
cursor.execute("SELECT * FROM countries WHERE LOWER(country_name) = LOWER(?);",
(eng_name.strip(),))
row = cursor.fetchone()
conn.close()
if row:
    return translate_db_record(dict(row), "country")
return None

def get_city_details(city_id):
    """Fetches details of a specific city by ID."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM cities WHERE id = ?;", (city_id,))
    row = cursor.fetchone()
    conn.close()
    if row:
        from utils.il8n import translate_db_record

        return translate_db_record(dict(row), "city")
    return None

def get_city_by_name(city_name):
    """Fetches details of a city by name (case-insensitive)."""
    from utils.il8n import get_english_term, translate_db_record

    eng_name = get_english_term(city_name)
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM cities WHERE LOWER(city_name) = LOWER(?);",
(eng_name.strip(),))
    row = cursor.fetchone()
    conn.close()
    if row:
        return translate_db_record(dict(row), "city")
    return None

def search_locations(query):
    """
    Searches for matching countries and cities.
    Returns a dict with key 'countries' and 'cities' containing matching rows.
    """
    if not query or not query.strip():
        return {"countries": [], "cities": []}

    from utils.il8n import get_english_term, translate_db_record

    eng_query = get_english_term(query)
    q = f"%{eng_query.strip()}%"
    conn = get_connection()
    cursor = conn.cursor()

    # Search countries
    cursor.execute(
        """
        SELECT * FROM countries
        WHERE country_name LIKE ? OR capital LIKE ? OR language LIKE ?;
        """,
```

```

        (q, q, q),
    )
    countries = [translate_db_record(dict(row), "country") for row in cursor.fetchall()]

    # Search cities
    cursor.execute(
        """
        SELECT cities.*, countries.country_name
        FROM cities
        JOIN countries ON cities.country_id = countries.id
        WHERE city_name LIKE ? OR description LIKE ? OR tourist_places LIKE ?;
        """,

        (q, q, q),
    )
    cities = [translate_db_record(dict(row), "city") for row in cursor.fetchall()]

    conn.close()
    return {"countries": countries, "cities": cities}

def get_weather_history(city_id):
    """Fetches all 12 months of weather data for a city, joined with city name, ordered
    calendar-wise."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        """
        SELECT weather_history.*, cities.city_name
        FROM weather_history
        JOIN cities ON weather_history.city_id = cities.id
        WHERE weather_history.city_id = ?
        ORDER BY weather_history.month_num ASC;
        """,
        (city_id,),
    )
    rows = cursor.fetchall()
    conn.close()
    from utils.i18n import translate_weather_record

    return [translate_weather_record(dict(row)) for row in rows]
# =====
# NEW USER AUTH & PROFILE PERSISTENCE METHODS
# =====

def create_user(full_name, email, phone, country, city, profile_pic, password_hash,
preferences="[]"):
    conn = get_connection()
    cursor = conn.cursor()
    try:
        cursor.execute(
            """
            INSERT INTO users (full_name, email, phone, country, city, profile_pic,
password_hash, preferences)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?);
            """,
            (full_name, email.strip().lower(), phone, country, city, profile_pic,
password_hash, preferences),
        )
        conn.commit()
        user_id = cursor.lastrowid
        conn.close()
        return user_id
    except sqlite3.IntegrityError:
        conn.close()

```

```
        return None

def get_user_by_email(email):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE LOWER(email) = LOWER(?);",
(email.strip(),))
    row = cursor.fetchone()
    conn.close()
    return dict(row) if row else None

def get_user_by_id(user_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE id = ?;", (user_id,))
    row = cursor.fetchone()
    conn.close()
    return dict(row) if row else None

def update_user_profile(user_id, full_name, phone, country, city, profile_pic,
preferences):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        """
        UPDATE users
        SET full_name = ?, phone = ?, country = ?, city = ?, profile_pic = ?, preferences =
?
        WHERE id = ?;
        """,
        (full_name, phone, country, city, profile_pic, preferences, user_id),
    )
    conn.commit()
    conn.close()
    return True

# =====
# ACTIVITY TRACKING & TRAVEL HISTORY LOGS
# =====

def log_activity(user_id, activity_type, query, details=None):
    import json

    if details and not isinstance(details, str):
        details = json.dumps(details)
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        """
        INSERT INTO travel_history (user_id, activity_type, query, details)
        VALUES (?, ?, ?, ?);
        """,
        (user_id, activity_type, query, details),
    )
    conn.commit()
    conn.close()
```

```
def get_travel_history(user_id, activity_type=None, search_query=None, start_date=None,
end_date=None, country=None):
    conn = get_connection()
    cursor = conn.cursor()

    query = "SELECT * FROM travel_history WHERE user_id = ?"
    params = [user_id]

    if activity_type:
        query += " AND activity_type = ?"
        params.append(activity_type)

    if search_query:
        query += " AND (query LIKE ? OR details LIKE ?)"
        params.append(f"%{search_query}%")
        params.append(f"%{search_query}%")

    if start_date:
        query += " AND date(created_at) >= date(?)"
        params.append(start_date)

    if end_date:
        query += " AND date(created_at) <= date(?)"
        params.append(end_date)

    if country:
        query += " AND (query LIKE ? OR details LIKE ?)"
        params.append(f"%{country}%")
        params.append(f"%{country}%")

    query += " ORDER BY created_at DESC"
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()
    return [dict(row) for row in rows]

def delete_history_item(history_id, user_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM travel_history WHERE id = ? AND user_id = ?;",
(history_id, user_id))
    conn.commit()

    conn.close()

def clear_user_history(user_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM travel_history WHERE user_id = ?;", (user_id,))
    conn.commit()
    conn.close()

def toggle_history_favorite(history_id, user_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT is_favorite FROM travel_history WHERE id = ? AND user_id =
?;", (history_id, user_id))
    row = cursor.fetchone()
    if row:
        new_val = 1 if row["is_favorite"] == 0 else 0
        cursor.execute(
            "UPDATE travel_history SET is_favorite = ? WHERE id = ? AND user_id = ?;",
(new_val, history_id, user_id))
```

```

        )
        conn.commit()
    conn.close()

# =====
# SAVED TRIPS & BOOKMARKS SYSTEMS
# =====

def save_trip(user_id, trip_type, name, collection_name, details, travel_date=None):
    import json

    if details and not isinstance(details, str):
        details = json.dumps(details)
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        """
        INSERT INTO saved_trips (user_id, trip_type, name, collection_name, details,
travel_date)
VALUES (?, ?, ?, ?, ?, ?);
        """,
        (user_id, trip_type, name, collection_name, details, travel_date),
    )
    conn.commit()
    conn.close()

def get_saved_trips(user_id, trip_type=None, collection_name=None):
    conn = get_connection()

    cursor = conn.cursor()
    query = "SELECT * FROM saved_trips WHERE user_id = ?"
    params = [user_id]
    if trip_type:
        query += " AND trip_type = ?"
        params.append(trip_type)
    if collection_name:
        query += " AND collection_name = ?"
        params.append(collection_name)
    query += " ORDER BY created_at DESC"
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()
    return [dict(row) for row in rows]

def delete_saved_trip(saved_trip_id, user_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM saved_trips WHERE id = ? AND user_id = ?;",
(saved_trip_id, user_id))
    conn.commit()
    conn.close()

# =====
# DASHBOARD ANALYTICS & STATS AGGREGATORS
# =====

def get_dashboard_stats(user_id):
    conn = get_connection()
    cursor = conn.cursor()

```

```

# 1. Total Trips Planned
cursor.execute(
    "SELECT COUNT(*) as count FROM saved_trips WHERE user_id = ? AND trip_type = 'itinerary';", (user_id,)
)
total_saved_itineraries = cursor.fetchone()["count"]

cursor.execute(
    "SELECT COUNT(*) as count FROM travel_history WHERE user_id = ? AND activity_type = 'itinerary';", (user_id,)
)
total_generated_itineraries = cursor.fetchone()["count"]

# 2. Countries Explored (distinct country keyword from history and saved)
cursor.execute(
    "SELECT DISTINCT query FROM travel_history WHERE user_id = ? AND activity_type IN ('search', 'itinerary');",
    (user_id,),
)
searches = [r["query"] for r in cursor.fetchall() if r["query"]]

cursor.execute("SELECT DISTINCT name FROM saved_trips WHERE user_id = ? AND trip_type = 'destination';", (user_id,))
destinations = [r["name"] for r in cursor.fetchall() if r["name"]]

countries_set = set()
for text in searches + destinations:
    t_lower = text.lower()
    if any(
        w in t_lower
        for w in [
            "india",
            "भारत",
            "ಭಾರತದೇಶಂ",
            "hyderabad",
            "visakhapatnam",
            "mumbai",
            "delhi",
            "bangalore",
            "chennai",
            "kolkata",
            "jaipur",
            "agra",
            "varanasi",
            "kochi",
            "goa",
            "udaipur",
            "pune",
            "ahmedabad",
            "amritsar",
            "srinagar",
            "shimla",
            "darjeeling",
            "mysore",
        ]
    ):
        countries_set.add("India")
    if any(
        w in t_lower
        for w in [
            "japan",
            "जापान",
            "ಜಪಾನ್",
            "tokyo",

```

```
        "osaka",
        "kyoto",
        "yokohama",
        "nagoya",
        "sapporo",
        "fukuoka",
        "kobe",

        "hiroshima",
        "nara",
        "okinawa",
        "kanazawa",
        "hakodate",
        "nagasaki",
        "sendai",
        "takayama",
        "himeji",
        "kamakura",
        "nikko",
        "matsumoto",
    ]
):
    countries_set.add("Japan")
if any(
    w in t_lower
    for w in [
        "singapore",
        "सिंगापुर",
        "సింగపూర్",
        "downtown",
        "sentosa",
        "orchard",
        "chinatown",
        "little india",
        "katong",
        "tampines",
        "jurong",
        "woodlands",
        "changi",
        "yishun",
        "ang mo kio",
        "bedok",
        "queenstown",
        "novena",
        "bukit timah",
        "punggol",
        "clementi",
        "serangoon",
    ]
):
    countries_set.add("Singapore")

countries_explored = len(countries_set)
if countries_explored == 0 and (total_saved_itineraries > 0 or
total_generated_itineraries > 0):
    countries_explored = 1

# 3. Saved Destinations Count
cursor.execute(

    "SELECT COUNT(*) as count FROM saved_trips WHERE user_id = ? AND trip_type =
'destination';", (user_id,)
)
saved_destinations = cursor.fetchone()["count"]
```

```
# 4. Recent Searches (top 5)
cursor.execute(
    "SELECT DISTINCT query, created_at FROM travel_history WHERE user_id = ? AND
activity_type = 'search' ORDER BY created_at DESC LIMIT 5;",
    (user_id,),
)
recent_searches = [dict(row) for row in cursor.fetchall()]

# 5. AI Usage Statistics
cursor.execute(
    "SELECT COUNT(*) as count FROM travel_history WHERE user_id = ? AND
activity_type = 'chat';", (user_id,)
)
chat_interactions = cursor.fetchone()["count"]

# 6. Upcoming Trips Timeline
cursor.execute(
    """
    SELECT * FROM saved_trips
    WHERE user_id = ? AND trip_type = 'itinerary' AND travel_date IS NOT NULL AND
travel_date != ''
    ORDER BY date(travel_date) ASC;
    """,
    (user_id,),
)
upcoming_trips = [dict(row) for row in cursor.fetchall()]

conn.close()
return {
    "total_trips": total_saved_itineraries + total_generated_itineraries,
    "countries_explored": countries_explored,
    "saved_destinations": saved_destinations,
    "recent_searches": recent_searches,
    "ai_usage": {"itineraries_generated": total_generated_itineraries,
"chatbot_interactions": chat_interactions},
    "upcoming_trips": upcoming_trips,
}
```

A.2 Cryptography and JWT Authentication Handler (utils/auth_utils.py)

```
import base64
import hashlib
import hmac
import json
import time
from typing import Optional

import streamlit as st

SECRET_KEY = "travelmate_super_secret_key_12345" # nsec B105

def hash_password(password: str) -> str:
    """Hashes a password using SHA-256 with a static salt for database storage."""
    salt = "travelmate_salt_6789"
    keyed = password + salt
    return hashlib.sha256(keyed.encode("utf-8")).hexdigest()

def verify_password(password: str, hashed: str) -> bool:
    """Verifies that a password matches its hashed form."""
    return hash_password(password) == hashed

def base64url_encode(payload) -> str:
    """Helper to base64url encode payloads or headers."""
    if isinstance(payload, dict):
        payload = json.dumps(payload).encode("utf-8")
    elif isinstance(payload, str):
        payload = payload.encode("utf-8")
    return base64.urlsafe_b64encode(payload).replace(b"=", b"").decode("utf-8")

def base64url_decode(s: str) -> bytes:
    """Helper to base64url decode strings."""
    padding = "=" * (4 - len(s) % 4)
    return base64.urlsafe_b64decode(s + padding)

def encode_jwt(payload: dict, expiry_seconds: int = 86400) -> str:
    """Generates a secure signed JWT-like token (HMAC-SHA256) for user sessions."""
    header = {"alg": "HS256", "typ": "JWT"}
    payload = payload.copy()
    payload["exp"] = int(time.time()) + expiry_seconds

    header_b64 = base64url_encode(header)
    payload_b64 = base64url_encode(payload)

    signature_input = f"{header_b64}.{payload_b64}".encode()

    signature = hmac.new(SECRET_KEY.encode("utf-8"), signature_input,
        hashlib.sha256).digest()
    signature_b64 = base64.urlsafe_b64encode(signature).replace(b"=", b"").decode("utf-8")

    return f"{header_b64}.{payload_b64}.{signature_b64}"

def decode_jwt(token: str) -> Optional[dict]:
    """Decodes and verifies a JWT-like token. Returns payload dict or None if
    invalid/expired."""
    if not token:
        return None
```

```

try:
    parts = token.split(".")
    if len(parts) != 3:
        return None
    header_b64, payload_b64, signature_b64 = parts

    # Verify signature
    signature_input = f"{header_b64}.{payload_b64}".encode()
    expected_signature = hmac.new(SECRET_KEY.encode("utf-8"), signature_input,
hashlib.sha256).digest()
    expected_signature_b64 =
base64.urlsafe_b64encode(expected_signature).replace(b"=", b"").decode("utf-8")

    if not hmac.compare_digest(signature_b64, expected_signature_b64):
        return None

    payload = json.loads(base64url_decode(payload_b64).decode("utf-8"))
    if payload.get("exp", 0) < time.time():
        return None # Expired

    return payload
except Exception:
    return None

def login_user(user_data: dict, remember_me: bool = False):
    """Sets the user session state and optionally a remember-me token."""
    # Strip sensitive data before putting user in session state
    session_user = {
        "id": user_data["id"],
        "full_name": user_data["full_name"],
        "email": user_data["email"],
        "phone": user_data.get("phone", ""),
        "country": user_data.get("country", ""),
        "city": user_data.get("city", ""),
        "profile_pic": user_data.get("profile_pic", ""),
        "preferences": user_data.get("preferences", []),
    }
    st.session_state.user = session_user
    token = encode_jwt(session_user, expiry_seconds=86400 * 30 if remember_me else
86400)
    st.session_state.auth_token = token
    if remember_me:

        # Mocking remember me by storing it in local storage / query params
        # In streamlit, we can also use query params or cookie mock in session state
        st.session_state.remember_me_token = token

def logout_user():
    """Logs the user out by clearing the session state."""
    st.session_state.user = None
    st.session_state.auth_token = None
    if "remember_me_token" in st.session_state:
        del st.session_state.remember_me_token
    # Clear chat history and other session states specific to user
    if "messages" in st.session_state:
        st.session_state.messages = []

```

A.3 Autonomous AI Agent and Tool Invoker (utils/chatbot_agent.py)

```

import json
import logging
import os
from typing import Any, Dict, Optional

logger = logging.getLogger(__name__)

# Check if google-adk is available and key is set
ADK_AVAILABLE = False
try:
    from google.adk.agents import Agent
    from google.adk.runners import Runner
    from google.adk.sessions import InMemorySessionService
    from google.genai import types  # noqa: F401

    ADK_AVAILABLE = True
except ImportError:
    logger.warning("google-adk package is not installed. Falling back to rule-based engine.")

from utils.database import (
    get_city_by_name,
    get_city_details,
    get_country_by_name,
    get_country_details,
    search_locations,
)

# -----
# Define Tools for Agent Kit
# Type hints and docstrings are crucial because ADK uses them
# to generate the JSON schemas for the model's function calls.
# -----

def get_country_info(country_name: str) -> Dict[str, Any]:
    """
    Retrieves complete travel details for a country by name.
    Use this to look up capital, currency, language, timezone, emergency numbers,
    visa regulations, traffic/legal rules, cultural etiquette, or safety guidelines.
    """
    country = get_country_by_name(country_name)
    if not country:
        return {"error": f"Country '{country_name}' not found in database."}
    return dict(country)

def get_city_info(city_name: str) -> Dict[str, Any]:
    """
    Retrieves detailed information for a city by name.
    Use this to look up city descriptions, transit guidelines, cuisines, dining,
    tourist spots/attractions, hotels, shopping areas, airport details, or safety
    warnings.
    """
    city = get_city_by_name(city_name)
    if not city:
        return {"error": f"City '{city_name}' not found in database."}

    # Safely parse JSON strings for food, places, and hotels
    res = dict(city)
    for field in ["food_info", "tourist_places", "hotel_info"]:
        if field in res and isinstance(res[field], str):
            try:

```

```

        res[field] = json.loads(res[field])
    except Exception: # nosec B110
        pass
    return res

def search_destinations(query: str) -> Dict[str, Any]:
    """
    Searches the travel database for matching countries or cities.
    Use this if the user is looking for destinations matching a keyword (like 'shrine'
    or 'beach').
    """
    return search_locations(query)

# -----
# Agent Kit Setup
# -----

_runner_instance: Optional[Any] = None

def get_agent_runner() -> Optional[Any]:
    global _runner_instance
    if not ADK_AVAILABLE:
        return None

    if _runner_instance is None:
        try:
            # Check for API Key in env or Streamlit secrets
            api_key = os.environ.get("GEMINI_API_KEY")
            if not api_key:
                # Try streamlit secrets if available
                try:
                    import streamlit as st

                    api_key = st.secrets.get("GEMINI_API_KEY")
                except Exception: # nosec B110
                    pass

            if not api_key:
                logger.warning("GEMINI_API_KEY not found. Agent Kit cannot be
initialized.")
                return None

            # Create Agent instance
            travel_agent = Agent(
                name="travel_mate_agent",
                model="gemini-2.5-flash",
                instruction=(
                    "You are TravelMate AI - a smart, helpful travel companion chatbot.
                    "
                    "Use the database tools provided (get_country_info, get_city_info,
                    search_destinations) "
                    "to answer user queries accurately. Answer clearly and structure
                    your responses with markdown. "
                    "If the user asks a question about food, rules, safety, attractions,
                    hotels, transit, or visa, "
                    "make sure to call the correct tool for the city/country they are
                    asking about. "
                    "If the user's query does not mention a country or city, refer to
                    the Active Context "
                    "provided in the system message or user prompt to determine the
                    relevant destination."
                ),

```

```

        tools=[get_country_info, get_city_info, search_destinations],
    )

    # Create Runner instance
    _runner_instance = Runner(agent=travel_agent,
session_service=InMemorySessionService())
    except Exception as e:
        logger.error(f"Failed to initialize Agent Kit: {e}")
        _runner_instance = None

    return _runner_instance

def is_agent_enabled() -> bool:
    """Returns True if google-adk is installed and GEMINI_API_KEY is configured."""
    if not ADK_AVAILABLE:
        return False
    return get_agent_runner() is not None

async def run_agent_query_async(
    user_id: str,
    session_id: str,
    query_text: str,
    active_country_id: Optional[int] = None,
    active_city_id: Optional[int] = None,
) -> str:
    """Runs the query using the ADK Agent Runner asynchronously."""
    runner = get_agent_runner()
    if not runner:
        raise RuntimeError("Agent runner is not initialized.")

    # Retrieve active context descriptions to feed to the agent
    context_str = ""

    if active_city_id:
        city = get_city_details(active_city_id)
        if city:
            context_str += f"Active City Context: {city['city_name']}. "
    if active_country_id:
        country = get_country_details(active_country_id)
        if country:
            context_str += f"Active Country Context: {country['country_name']}. "

    # Prepend active context to guide the agent in case of general queries
    full_prompt = query_text
    if context_str:
        full_prompt = f"[{context_str.strip()}]\n{query_text}"

    content = types.Content(role="user", parts=[types.Part(text=full_prompt)])

    final_response = ""
    async for event in runner.run_async(user_id=user_id, session_id=session_id,
new_message=content):
        if event.is_final_response():
            if event.content and event.content.parts:
                final_response = event.content.parts[0].text
                break

    if not final_response:
        final_response = "I couldn't process the query with the Agent. Please try
again."

    return final_response

```

```
def run_agent_query(
    user_id: str,
    session_id: str,
    query_text: str,
    active_country_id: Optional[int] = None,
    active_city_id: Optional[int] = None,
) -> str:
    """Synchronous entry point to run the agent query."""
    import asyncio
    from concurrent.futures import ThreadPoolExecutor

    coro = run_agent_query_async(user_id, session_id, query_text, active_country_id,
    active_city_id)
    try:
        # Check if an event loop is already running in this thread
        loop = asyncio.get_running_loop()
    except RuntimeError:
        loop = None

    if loop and loop.is_running():
        # Run in a separate thread so we don't block the main event loop
        with ThreadPoolExecutor(max_workers=1) as executor:

            future = executor.submit(asyncio.run, coro)
            return future.result()
    else:
        return asyncio.run(coro)
```

A.4 User Interface View for AI Travel Assistant (pages/chatbot.py)

```

import json
import time

import streamlit as st

from utils.chatbot_agent import is_agent_enabled, run_agent_query
from utils.database import (
    get_all_countries,
    get_cities_by_country,
    get_city_details,
    get_country_details,
    get_weather_history,
)
from utils.il18n import get_english_term, translate_ui
from utils.styles import render_hero

render_hero(translate_ui("ai_chatbot_title"), translate_ui("ai_chatbot_subtitle"))

# AI Engine status indicator
if is_agent_enabled():
    st.markdown(
        """
        <div style='display:inline-flex; align-items:center; gap:8px;
        background:rgba(0,200,100,0.1); border:1px solid rgba(0,200,100,0.25); border-
        radius:100px; padding:6px 16px; margin-bottom:20px;'>
            <span style='width:8px; height:8px; background:#00c864; border-radius:50%;
            display:inline-block; box-shadow:0 0 8px #00c864;'></span>
            <span style='font-size:0.85rem; font-weight:700; color:#00c864;'>Agent Kit
            AI Engine Active (Autonomous Tools)</span>
        </div>
        """,
        unsafe_allow_html=True,
    )
else:
    st.markdown(
        """
        <div style='display:inline-flex; align-items:center; gap:8px;
        background:rgba(128,128,128,0.1); border:1px solid rgba(128,128,128,0.2); border-
        radius:100px; padding:6px 16px; margin-bottom:20px;'>
            <span style='width:8px; height:8px; background:gray; border-radius:50%;
            display:inline-block;'></span>
            <span style='font-size:0.85rem; font-weight:700; color:gray;'>Offline Mode
            Active (Rule-based Fallback)</span>
        </div>
        """,
        unsafe_allow_html=True,
    )

# Helper function to match keywords and generate context-rich answers
def generate_bot_response(user_query, active_country_id, active_city_id):
    # Normalize user_query to English keywords
    eng_query = get_english_term(user_query)
    query = eng_query.lower().strip()
    import re
    tokens = set(re.findall(r"\b[a-z0-9-]+\b", query))

    # 1. Database details for active session state (context fallbacks)

    active_city = get_city_details(active_city_id) if active_city_id else None
    active_country = get_country_details(active_country_id) if active_country_id else None
    None

    # 2. Extract country/city names from query

```

```

    matched_country = None
    matched_city = None

    countries = get_all_countries()
    for c in countries:
        # Match against either English query or raw user query (translated country
names)
        if c["country_name"].lower() in query or c["country_name"].lower() in
user_query.lower():
            matched_country = c
            break

    # List all cities in DB
    for c in countries:
        cities = get_cities_by_country(c["id"])
        for ct in cities:
            if ct["city_name"].lower() in query or ct["city_name"].lower() in
user_query.lower():
                matched_city = ct
                break

    # 3. Determine target entity (City has priority, then Country, then active context)
    target_city = matched_city if matched_city else (active_city if not matched_country
else None)
    target_country = matched_country if matched_country else (active_country if not
target_city else None)

    # If target city is set, get its country details too
    if target_city and not target_country:
        target_country = get_country_details(target_city["country_id"])

    # Topic detection (using tokens to prevent substring matches like "eat" in
"weather")
    is_food = any(w in tokens for w in ["food", "eat", "try", "dish", "cuisine",
"delicacies", "dining", "lunch", "dinner", "biryani", "sushi", "crab", "ramen"])
    is_rules = any(w in tokens for w in ["rule", "rules", "law", "laws", "traffic",
"regulation", "regulations", "fine", "fines", "forbidden", "prohibit", "prohibited",
"smoking"]) or "chewing gum" in query
    is_etiquette = any(w in tokens for w in ["etiquette", "culture", "do", "don't",
"dont", "dos", "dents", "respect", "bow", "tip", "tipping", "shoe", "shoes", "hand",
"hands", "custom", "customs"])
    is_safety = any(w in tokens for w in ["safety", "safe", "scam", "scams", "crime",
"crimes", "emergency", "police", "ambulance", "water", "drinkable"]) or "drinking water"
in query
    is_attraction = any(w in tokens for w in ["place", "places", "attraction",
"attractions", "visit", "see", "sightseeing", "temple", "temples", "shrine", "shrines",
"monument", "monuments", "tour", "tours"])
    is_hotel = any(w in tokens for w in ["hotel", "hotels", "stay", "stays",
"accommodation", "accommodations", "resort", "resorts", "hostel", "hostels"])
    is_transport = any(w in tokens for w in ["transport", "transit", "bus", "buses",
"metro", "subway", "train", "trains", "taxi", "taxis", "airport", "airports"])
    is_visa = any(w in tokens for w in ["visa", "entry", "passport", "arrival"])
    is_weather = any(w in tokens for w in ["weather", "climate", "temp", "temperature",
"rain", "rainfall", "monsoon", "season", "seasons", "forecast", "hot", "cold", "sunny"])
or "best time" in query

    # Build response based on match
    if target_city:
        city_name = target_city["city_name"]
        country_name = target_country["country_name"] if target_country else ""

        if is_food:
            try:
                foods = json.loads(target_city["food_info"])
                food_list = "\n".join([f"- **{f['name']}** ({f['type']}): {f['desc']}"]

```

```

for f in foods])

        return translate_ui("bot_food_city").format(city=city_name,
country=country_name, list=food_list)
    except Exception:
        return f"🍽️ Stored food info for {city_name}:
{target_city['food_info']}"

    elif is_attraction:
        try:
            places = json.loads(target_city["tourist_places"])
            place_list = "\n".join(
                [
                    f"- **{p['name']}** ({p.get('rating', '4.5')}) - {p['desc']}
Best visited in the *{p.get('time', 'day')}*."
                    for p in places
                ]
            )
            return translate_ui("bot_attract_city").format(city=city_name,
list=place_list)
        except Exception:
            return f"🏰 Stored attractions for {city_name}:
{target_city['tourist_places']}"

    elif is_hotel:
        try:
            hotels = json.loads(target_city["hotel_info"])
            hotel_list = ""
            for tier, details in hotels.items():
                hotel_list += (
                    f"- **{tier.upper()}**: {details['name']} ({details.get('price',
''))} - *{details['desc']}*\n"
                )
            return translate_ui("bot_hotel_city").format(city=city_name,
list=hotel_list)
        except Exception:
            return f"🏨 Stored hotels for {city_name}: {target_city['hotel_info']}"

    elif is_transport:
        return translate_ui("bot_transit_city").format(
            city=city_name, transit=target_city["transport_info"],
airport=target_city["airport_details"]
        )

    elif is_safety:
        return translate_ui("bot_safety_city").format(city=city_name,
safety=target_city["safety_recommendations"])

    elif is_rules or is_etiquette:
        # Fallback to country rules since rules are usually country-wide
        return translate_ui("bot_rules_city").format(
            city=city_name,
            country=country_name,
            rules=target_country["rules"],
            etiquette=target_country["etiquette"],
        )
    elif is_weather:
        try:
            history = get_weather_history(target_city["id"])
            if history:
                # Try to find a month name mentioned in the query

                target_month_num = None
                months_eng = ["january", "february", "march", "april", "may",
"june", "july", "august", "september", "october", "november", "december"]

```

```

        for idx, m in enumerate(months_eng):
            if m in query or m[:3] in query:
                target_month_num = idx + 1
                break

        if target_month_num is not None:
            curr_weather = next((row for row in history if row["month_num"]
== target_month_num), history[0])
        else:
            import datetime
            curr_month_num = datetime.datetime.now().month
            curr_weather = next((row for row in history if row["month_num"]
== curr_month_num), history[0])

        return translate_ui("bot_weather_city").format(
            city=city_name,
            country=country_name,
            desc=f"{curr_weather['month_name']} -
{curr_weather['description']}",
            temp=curr_weather["avg_temp"],
            rain=curr_weather["rainfall"],
            recommendation=curr_weather["recommendation"],
        )
    else:
        return f"🌩️ Weather data for {city_name} is currently not
available."
    except Exception as e:
        return f"🌩️ Weather data for {city_name} could not be loaded: {e}"

    else:
        # General city info response
        return translate_ui("bot_welcome_city").format(city=city_name,
desc=target_city["description"])

    elif target_country:
        country_name = target_country["country_name"]

        if is_rules:
            rules_formatted = "\n".join([f"- {r}" for r in
target_country["rules"].split("\n") if r.strip()])
            return translate_ui("bot_rules_country").format(country=country_name,
rules=rules_formatted)

        elif is_etiquette:
            et_formatted = "\n".join([f"- {e}" for e in
target_country["etiquette"].split("\n") if e.strip()])
            return translate_ui("bot_etiquette_country").format(country=country_name,
etiquette=et_formatted)

        elif is_safety:
            safety_formatted = "\n".join([f"- {s}" for s in
target_country["safety_tips"].split("\n") if s.strip()])
            return translate_ui("bot_safety_country").format(
                country=country_name, safety=safety_formatted,
                emergency=target_country["emergency_number"]
            )

        elif is_visa:
            return translate_ui("bot_visa_country").format(country=country_name,
visa=target_country["visa_info"])

    elif is_weather:
        cities_in_c = get_cities_by_country(target_country["id"])
        if cities_in_c:
            capital_name = target_country.get("capital", "").lower()
            matching_cities = [c for c in cities_in_c if c["city_name"].lower() ==

```

```

capital_name]
target_city_obj = matching_cities[0] if matching_cities else
cities_in_c[0]
try:
    history = get_weather_history(target_city_obj["id"])
    if history:
        import datetime
        curr_month_num = datetime.datetime.now().month
        curr_weather = next((row for row in history if row["month_num"]
== curr_month_num), history[0])
        return f"🌤️ **Weather in {target_country['country_name']}**
(showing details for the capital/city: **{target_city_obj['city_name']}**):\n\n" + \
            translate_ui("bot_weather_city").format(
                city=target_city_obj["city_name"],
                country=target_country["country_name"],
                desc=f"{curr_weather['month_name']} -
{curr_weather['description']}",
                temp=curr_weather["avg_temp"],
                rain=curr_weather["rainfall"],
                recommendation=curr_weather["recommendation"],
            )
    except Exception:
        pass
    return f"🌤️ Weather data for {country_name} is not directly available.
Please ask about a specific city like Tokyo, Mumbai, or Singapore."

else:
    # General country info response
    return translate_ui("bot_welcome_country").format(
        country=country_name,
        capital=target_country["capital"],
        currency=target_country["currency"],
        language=target_country["language"],
        timezone=target_country["timezone"],
    )

# Fallback when no keywords or locations matched
return translate_ui("bot_fallback")

# Clear chat history if language changes (ensures consistent language context)
if "chat_language" not in st.session_state:
    st.session_state.chat_language = st.session_state.get("language", "en")

if "messages" not in st.session_state or st.session_state.chat_language !=
st.session_state.get("language", "en"):
    st.session_state.messages = [{"role": "assistant", "content":
translate_ui("chat_bot_welcome")}]
    st.session_state.chat_language = st.session_state.get("language", "en")

# Display quick suggest options
st.markdown(translate_ui("suggested_questions"))
sug_coll1, sug_coll2, sug_coll3 = st.columns(3)

with sug_coll1:
    if st.button(translate_ui("suggest_hyd_food"), use_container_width=True):
        st.session_state.prompt_trigger = "What food should I try in Hyderabad?"
with sug_coll2:
    if st.button(translate_ui("suggest_sg_rules"), use_container_width=True):
        st.session_state.prompt_trigger = "What are the rules in Singapore?"
with sug_coll3:
    if st.button(translate_ui("suggest_tokyo_etiq"), use_container_width=True):
        st.session_state.prompt_trigger = "What is the cultural etiquette in Tokyo?"

# Process quick triggers

```

```

user_prompt = None
if "prompt_trigger" in st.session_state and st.session_state.prompt_trigger:
    user_prompt = st.session_state.prompt_trigger
    st.session_state.prompt_trigger = None # Reset trigger
else:
    # Get user input from chat input box
    user_prompt = st.chat_input(translate_ui("chat_placeholder"))

# Render chat history
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# Process new input
if user_prompt:
    # Display user query
    with st.chat_message("user"):
        st.markdown(user_prompt)
    st.session_state.messages.append({"role": "user", "content": user_prompt})

    # Generate response
    if is_agent_enabled():
        try:
            with st.spinner("TravelMate AI is thinking..."):
                response = run_agent_query(
                    user_id=st.session_state.user["email"] if
st.session_state.get("user") else "guest_user",
                    session_id=st.session_state.get("chat_language", "en"),
                    query_text=user_prompt,
                    active_country_id=st.session_state.get("selected_country_id"),
                    active_city_id=st.session_state.get("selected_city_id"),
                )
            except Exception as e:
                st.warning(f"Agent Kit encountered an error: {e}. Falling back to rule-based
response.")
                response = generate_bot_response(
                    user_prompt, st.session_state.get("selected_country_id"),
st.session_state.get("selected_city_id")
                )
        else:
            response = generate_bot_response(

                user_prompt, st.session_state.get("selected_country_id"),
st.session_state.get("selected_city_id")
            )

    # Auto-Log Chat Activity
    if st.session_state.get("user"):
        from utils.database import log_activity

        log_activity(st.session_state.user["id"], "chat", user_prompt, {"response":
response})

    # Display assistant response with a simulated typing speed
    with st.chat_message("assistant"):
        message_placeholder = st.empty()
        full_response = ""
        # Simulate stream typing
        for chunk in response.split(" "):
            full_response += chunk + " "
            time.sleep(0.04) # brief delay
            message_placeholder.markdown(full_response + "▮")
        message_placeholder.markdown(full_response)

    st.session_state.messages.append({"role": "assistant", "content": response})

```


Appendix B: Detailed SQL Schema Definitions

The following represents the complete SQL schema DDL scripts executed by `init_db()` to construct the relational tables:

```
CREATE TABLE IF NOT EXISTS countries (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    country_name TEXT UNIQUE NOT NULL,  
    capital TEXT NOT NULL,  
    currency TEXT NOT NULL,  
    language TEXT NOT NULL,  
    timezone TEXT NOT NULL,  
    emergency_number TEXT NOT NULL,  
    visa_info TEXT NOT NULL,  
    rules TEXT NOT NULL,  
    etiquette TEXT NOT NULL,  
    safety_tips TEXT NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS cities (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    country_id INTEGER NOT NULL,  
    city_name TEXT NOT NULL,  
    description TEXT NOT NULL,  
    transport_info TEXT NOT NULL,  
    food_info TEXT NOT NULL,  
    tourist_places TEXT NOT NULL,  
    hotel_info TEXT NOT NULL,  
    shopping_areas TEXT NOT NULL,  
    airport_details TEXT NOT NULL,  
    safety_recommendations TEXT NOT NULL,  
    FOREIGN KEY (country_id) REFERENCES countries (id) ON DELETE CASCADE,  
    UNIQUE(country_id, city_name)  
);  
  
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    full_name TEXT NOT NULL,  
    email TEXT UNIQUE NOT NULL,  
    phone TEXT,  
    country TEXT,  
    city TEXT,  
    profile_pic TEXT,  
    password_hash TEXT NOT NULL,  
    preferences TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Appendix C: Test Execution & Spec Verifications

The following BDD spec-style verification scenarios are executed via pytest tests/ to validate chatbot responses and context engine fallback logic:

```
def describe_chatbot_response():
    def it_handles_greeting_fallback_when_no_match(monkeypatch, tmp_path):
        setup_temp_db(monkeypatch, tmp_path)
        response = generate_bot_response("hello companion", None, None)
        assert "Hello! I am your AI Travel Companion" in response

    def it_responds_with_food_specials_for_matched_city(monkeypatch, tmp_path):
        setup_temp_db(monkeypatch, tmp_path)
        response = generate_bot_response("What food should I try in Hyderabad?", None,
None)
        assert "must-try foods" in response
        assert "Hyderabad" in response

    def it_falls_back_to_active_city_context_for_food(monkeypatch, tmp_path):
        setup_temp_db(monkeypatch, tmp_path)
        hyd = db.get_city_by_name("Hyderabad")
        response = generate_bot_response("Tell me what food to try", None, hyd["id"])
        assert "must-try foods in Hyderabad" in response
```